



Nombre: Grilmar Adrian

Apellidos: Marroquin Vargas

Carne: 202506479

Facultad: Ingeniería en Sistemas

Fecha: 16/05/2026

Tema: Examen Final

Árbol Binario de Búsqueda (BST)

Identificación: Árbol binario donde para cada nodo, claves menores van a la izquierda y mayores a la derecha. Complejidad (Big-O):

- **Búsqueda:** Peor caso $O(n)$; Promedio $O(\log n)$
- **Inserción:** Peor caso $O(n)$; Promedio $O(\log n)$
- **Eliminación:** Peor caso $O(n)$; Promedio $O(\log n)$
- **Características mecánicas:** Estructura binaria sin balanceo automático; orden mantenido por comparación recursiva/iterativa.
- **Ventajas:** Simple de implementar; eficiente si está balanceado.
- **Limitaciones:** Degenera a lista enlazada con entradas ordenadas (peor caso).
- **Caso de uso:** Índices en memoria para búsquedas rápidas en aplicaciones con datos dinámicos.

Árbol Balanceado (AVL)

Identificación: BST auto-balanceado que mantiene factor de equilibrio (altura) en cada nodo. Complejidad (Big-O):

- **Búsqueda:** $O(\log n)$ (peor y promedio)
- **Inserción:** $O(\log n)$ (peor y promedio)
- **Eliminación:** $O(\log n)$ (peor y promedio)
- **Características mecánicas:** Rota subárboles (rotaciones simples/dobles) para mantener diferencia de alturas ≤ 1 .
- **Ventajas:** Garantiza alturas logarítmicas; búsquedas rápidas.
- **Limitaciones:** Rotaciones adicionales en inserción/eliminación; mayor sobrecarga en escrituras.
- **Caso de uso:** Estructuras en memoria donde se requiere latencia de búsqueda garantizada (bases de datos en memoria, caches).

Árbol B / B+

Identificación: Árbol multi-way balanceado optimizado para almacenamiento en disco; B+ almacena claves en hojas enlazadas.

- **Complejidad (Big-O):**
- **Búsqueda:** $O(\log_m n)$ donde m es el orden (peor y promedio)
- **Inserción:** $O(\log_m n)$
- **Eliminación:** $O(\log_m n)$
- **Características mecánicas:** Nodos con múltiples hijos; alto factor de ramificación reduce altura; diseñado para minimizar accesos a disco.
- **Ventajas:** Excelente para sistemas de archivos y bases de datos en disco; lecturas secuenciales eficientes (B+).
- **Limitaciones:** Implementación más compleja; overhead en memoria para punteros y splits/merges.
- **Caso de uso:** Índices en motores de bases de datos relacionales (ej. PostgreSQL usa variantes de B-trees).

Trie (Árbol de Prefijos)

Identificación: Árbol en el que cada arista representa un carácter; útil para almacenar conjuntos de cadenas.

- **Complejidad (Big-O):**
- **Búsqueda:** $O(L)$ donde L es la longitud de la clave (peor y promedio)
- **Inserción:** $O(L)$
- **Eliminación:** $O(L)$
- **Características mecánicas:** Cada nivel corresponde a posición en la cadena; nodos pueden almacenar marcador de fin de palabra.
- **Ventajas:** Búsqueda por prefijo muy rápida; útil para autocompletado y diccionarios.
- **Limitaciones:** Alto consumo de memoria si no se usa compresión (p. ej. radix trie).
- **Caso de uso:** Autocompletado, motores de búsqueda de prefijos, tablas de enrutamiento.

Heap (Montículo Min/Max)

Identificación: Árbol binario completo que satisface la propiedad de heap (padre $\leq$ hijos para min-heap).

- **Complejidad (Big-O):**
- **Búsqueda (valor arbitrario):** $O(n)$
- **Insertar (push):** $O(\log n)$
- **Eliminar raíz (pop):** $O(\log n)$
- **Características mecánicas:** Representación eficiente en arreglo; mantiene orden parcial (no BST).
- **Ventajas:** Acceso rápido al mínimo/máximo; ideal para colas de prioridad.
Limitaciones: No es eficiente para búsquedas arbitrarias; no mantiene orden total.
- **Caso de uso:** Planificadores de tareas, algoritmos de Dijkstra, colas de prioridad.